Project BlueLine — Project Handover Document

Prepared for: PGE, Core & Main, ATCO Prepared by: LTM AI-Led Engineering Team Version: 1.0 | Date: April 2026 Status: Delivered

---

Table of Contents

1. What Was Built
2. The Three Automation Tracks
3. Agent Catalogue — All 12 Agents
4. How Each Track Works
5. What You Get — Deliverables
6. Azure Infrastructure Deployed
7. System Integrations
8. Human Control Points
9. Getting Started — What We Need From You
10. Operating the System Day-to-Day
11. Monitoring and Alerts
12. Cost Estimate
13. Handover Checklist
14. Support and Ownership After Handover
15. Frequently Asked Questions

---

1. What Was Built

Project BlueLine is an AI-powered automation system built for your engineering team. It replaces three manual, time-consuming workflows with intelligent agents that run automatically in the background — without changing how your developers work.

The Three Problems It Solves

| Problem | Before BlueLine | After BlueLine | |---|---|---| | **Code Review** | Reviewers manually check every PR against .NET and Angular standards — slow, inconsistent, reviewer-dependent | AI agents review every

PR the moment it is opened, post inline comments, and flag risks — reviewer focuses on logic, not checklist | | **Security (Fortify)** | Fortify scans produce a list of findings — engineers manually read, triage, research, and fix each one | AI agents triage every finding, rank by severity, and generate the actual fix code as a draft PR | | **Certificate Renewal** | SSL certificates tracked via spreadsheet — manual requests, manual installs across Dev, QA, Prod | System detects expiry automatically, requests renewal, and deploys to all environments — zero manual steps |

Core Design Principle

Humans stay in control. BlueLine is designed to assist, not replace. Every important action — merging code, deploying to Production — requires a human to approve. Agents do the analysis and preparation work; your team makes the decisions.

---

2. The Three Automation Tracks

```
┌──────────────────────────────────────────────────────────────┐
│                      PROJECT BLUELINE                        │
│                                                              │
│   ┌────────────────┐  ┌────────────────┐  ┌──────────────┐  │
│   │ QUALITY GATE   │  │ SECURITY LOOP  │  │ CERT LOOP    │  │
│   │                │  │                │  │              │  │
│   │ Triggered by:  │  │ Triggered by:  │  │ Triggered by │  │
│   │ PR opened      │  │ Pipeline run   │  │ Daily timer  │  │
│   │ or updated     │  │ or schedule    │  │              │  │
│   │                │  │                │  │              │  │
│   │ 4 Agents       │  │ 4 Agents       │  │ 4 Agents     │  │
│   └────────────────┘  └────────────────┘  └──────────────┘  │
└──────────────────────────────────────────────────────────────┘
```

Track 1 — Quality Gate (Code Review)

Fires every time a developer opens or updates a Pull Request in Azure DevOps. Three agents analyse the code in parallel and a fourth aggregates their findings into one consolidated review comment on the PR.

Track 2 — Security Loop (Fortify)

Fires when the CI/CD pipeline runs, or on a scheduled scan. Agents fetch findings from Fortify SSC, triage each one with AI, and generate fix code as a draft PR for confirmed vulnerabilities.

Track 3 — Certificate Loop (SSL Renewal)

Runs every morning on a timer. Agents check Azure Key Vault for certificates expiring within 30 days, raise renewal requests automatically, and deploy the renewed certificate to all environments — with a human approval gate before Production.

---

3. Agent Catalogue — All 12 Agents

Quality Gate Track

| Agent | What It Does | Output | |---|---|---| | **CLARION** | Checks every changed file in the PR against your .NET and Angular coding standards — naming conventions, structure, patterns, security smells | Inline PR comments with the exact line, the violation, and the corrected code | | **LUMEN** | Detects code smells and anti-patterns — dead code, overly complex methods, coupling issues, type safety violations | Annotated findings with plain-language explanations of why it matters | | **VECTOR** | Scores each file by risk and complexity — identifies which parts of the PR need the most reviewer attention | Per-file risk score; reviewer attention flags for high-risk areas | | **ASCENT** | Reads the output from CLARION, LUMEN, and VECTOR and produces one final recommendation | A single PR summary comment: APPROVE / REQUEST CHANGES / BLOCK — with prioritised finding list |

Security Loop Track

| Agent | What It Does | Output | |---|---|---| | **WATCHTOWER** | Monitors Fortify SSC for new findings on a schedule; triggers the security pipeline when new findings appear | Alert with list of new findings ready for triage | | **BULWARK** | Takes each Fortify finding and classifies it using AI — CRITICAL, HIGH, NEEDS REVIEW, or FALSE POSITIVE — with reasoning | Classified finding list with context and remediation priority | | **FORGE** | For every CRITICAL or HIGH finding, reads the source file, generates a correct fix, and creates a draft PR in Azure DevOps | Draft fix PR with the corrected code and a test | | **STEWARD** | Logs every agent decision to an immutable audit trail — what was found, how it was classified, what action was taken, by whom | Structured audit log entries in Azure Blob Storage (permanent, tamper-evident) |

Certificate Loop Track

| Agent | What It Does | Output | |---|---|---| | **TIMELINE** | Queries Azure Key Vault daily for all certificate expiry dates; flags any certificate expiring within 30 days | Renewal work items for each at-risk certificate | | **REGENT** | Maintains a structured inventory of all certificates — name, owner, environment, CA, expiry date | Updated certificate inventory in Azure Table Storage | | **COURIER** | Raises the certificate renewal request with the Certificate Authority (internal C&M portal or external CA) and downloads the issued certificate | Renewed certificate file, validated for correctness | | **HARBOUR** | Installs the renewed

certificate on the target server — IIS or Azure App Service — across Dev, QA, and Prod; verifies HTTPS is working after each install | Deployment confirmation + HTTPS verification result per environment |

---

4. How Each Track Works

4.1 Quality Gate — Step by Step

```
Developer opens a Pull Request in Azure DevOps
                    |
                    ▼
        Azure DevOps sends webhook to BlueLine
                    |
          ┌─────────┴──────────┐
          ▼         ▼          ▼
       CLARION    LUMEN      VECTOR
   (standards) (smells)    (risk)
      run at the same time (~30-45 sec)
          |         |          |
          └─────────┴──────────┘
                    |
                    ▼
                ASCENT
        aggregates all findings
        posts ONE summary comment on the PR:
        ┌──────────────────────────────┐
        │  RECOMMENDATION: REQUEST CHANGES│
        │  Critical: 2 | Medium: 3       │
        │  Reviewer must check: File A   │
        │  Risk score: 7.2/10            │
        └──────────────────────────────┘
                    |
                    ▼
        Human reviewer sees pre-annotated PR
        Focuses on logic — not the checklist
                    |
                    ▼
        Human APPROVES or REQUESTS CHANGES
        (agents cannot merge — ever)
```

Time from PR open to first comment: 20–45 seconds.

4.2 Security Loop — Step by Step

```
CI/CD pipeline runs (or WATCHTOWER scheduled scan fires)
                    │
                    ▼
          WATCHTOWER detects new Fortify findings
                    │
                    ▼
          BULWARK triages each finding:
          ┌──────────────────────────────────┐
          │  SQL Injection → CRITICAL        │
          │  Unused variable → FALSE POSITIVE │
          │  Insecure config → HIGH          │
          │  Ambiguous pattern → NEEDS REVIEW │
          └──────────────────────────────────┘
                    │
          CRITICAL/HIGH findings ──────────────────────┐
                    │                                  │
                    ▼                                  ▼
          FORGE reads the source file          STEWARD logs everything
          Generates the fix code               to audit trail
          Creates a draft PR:
          ┌──────────────────────────────────┐
          │  Branch: fix/sql-injection-line-142 │
          │  Fix: parameterized query applied  │
          │  Test: unit test added             │
          │  Status: DRAFT — awaiting approval │
          └──────────────────────────────────┘
                    │
                    ▼
          Human reviews and approves the fix PR
          (agents cannot merge code — ever)
```

4.3 Certificate Loop — Step by Step

```
Every morning — TIMELINE runs
                    │
                    ▼
          Queries Azure Key Vault for all certs
          Flags any expiring within 30 days:
          ┌──────────────────────────────────┐
```

```
        │  api.coreandmain.com  → 12 days left  │
        │  payments.pge.com     → 28 days left  │
        └──────────────────────────────────────┘
                  │
                  ▼
        REGENT updates the certificate inventory
                  │
                  ▼
        COURIER raises renewal request with CA
        Downloads renewed certificate
        Validates: expiry ✅  domain ✅  chain ✅
                  │
                  ▼
        HARBOUR deploys automatically:
        Dev  ─── install ─── verify HTTPS ✅
        QA   ─── install ─── verify HTTPS ✅
        Prod ─── HOLDS ──────────────────────┐
                                             │
                  ┌──────────────────────────┘
                  ▼
        Teams notification sent to approver:
        ┌──────────────────────────────────────┐
        │  Certificate ready for Prod deployment  │
        │  Domain: api.coreandmain.com            │
        │  New expiry: 2027-04-27                 │
        │  [APPROVE DEPLOYMENT]  [REJECT]         │
        └──────────────────────────────────────┘
                  │
                  ▼
        Human clicks APPROVE
        HARBOUR deploys to Prod
        Verifies HTTPS ✅
        STEWARD writes audit log entry
```

---

5. What You Get — Deliverables

Code Delivered

| Component | Location | Description | |---|---|---| | All 12 agents | `poc/agents/` | Python source for CLARION, LUMEN, VECTOR, ASCENT, BULWARK, WATCHTOWER, FORGE, STEWARD, TIMELINE, REGENT, COURIER, HARBOUR | | Base agent framework | `poc/core/base_agent.py` | Shared agent foundation — logging, LLM client, escalation, audit | | Azure DevOps client | `poc/utils/azure_devops.py` | REST API

client for PR comments, branch creation, work items | | LLM client | `poc/utils/llm_client.py` | Azure OpenAI / Anthropic API connector with prompt caching | | PR orchestrator | `poc/utils/pr_runner.py` | Runs all Quality Gate agents on a real PR | | Demo application | `poc/app.py` | Streamlit UI for demonstrating all agents | | DAS standards doc | `docs/das_review_standards.md` | Coding standards loaded into CLARION and LUMEN | | Sample files | `poc/samples/` | C#, TypeScript, Fortify finding samples for demo | | Azure infrastructure | `infrastructure/` | Bicep templates for all Azure resources |

Documents Delivered

| Document | Purpose | |---|---| | `BlueLine_Understanding_Document.md` | Full project scope, objectives, risks, rollout plan | | `BlueLine_LLD.md` | Low-level technical design — every agent, every API, every data model | | `BlueLine_Agent_Architecture_Overview.md` | How agents are built — for engineers extending the system | | `BlueLine_POC_Setup_Guide.md` | Step-by-step guide to run the POC locally | | `BlueLine_POC_Test_Cases.md` | Test cases for validating every agent | | `BlueLine_RnD_Research_Document.md` | Why this approach was chosen over alternatives | | `BlueLine_Discovery_Questions.md` | Discovery questions for production onboarding | | `AI_Tools_Overview_CodeReview_Security.md` | Market tool comparison and recommendations | | This document | Client handover — what was built, how to use it |

---

6. Azure Infrastructure Deployed

All BlueLine components run inside **your Azure tenant**. No data leaves your environment.

Resource Groups

| Resource Group | Contains | |---|---| | `blueline-rg-shared` | Key Vault, Service Bus, Storage Account, Log Analytics | | `blueline-rg-quality` | Quality Gate Function App (CLARION, LUMEN, VECTOR, ASCENT) | | `blueline-rg-security` | Security Loop Function App (WATCHTOWER, BULWARK, FORGE, STEWARD) | | `blueline-rg-certloop` | Certificate Loop Function App (TIMELINE, REGENT, COURIER, HARBOUR) |

Key Azure Resources

| Resource | Type | Purpose | |---|---|---| | `blueline-quality-fa` | Azure Function App (Premium) | Runs Quality Gate agents — always warm for fast PR response | | `blueline-security-fa` | Azure Function App (Consumption) | Runs Security agents — triggered by pipeline or schedule | | `blueline-certloop-fa` | Azure Function App (Consumption) | Runs Certificate agents — runs daily | | `blueline-kv` | Azure Key Vault | Stores API keys, CA credentials, certificates | | `blueline-sb` | Azure Service Bus | Agent-to-agent message passing | | `blueline-storage` | Azure Storage Account | Agent state and STEWARD audit logs | | `blueline-law` | Log Analytics Workspace | All agent logs, alerts, dashboards | | `blueline-apim` | Azure API Management | Webhook ingress and authentication |

Technology Stack Summary

| Layer | Technology | |---|---| | Agent runtime | Azure Functions — Python 3.11 | | AI reasoning | Anthropic Claude (`claude-sonnet-4-6`) via Azure OpenAI | | Agent communication | Azure Service Bus | | Secret storage | Azure Key Vault | | Certificate storage | Azure Key Vault Certificates | | Audit logs | Azure Blob Storage (immutable) | | Monitoring | Azure Monitor + Log Analytics | | Notifications | Microsoft Teams webhooks | | Infrastructure as Code | Azure Bicep |

---

7. System Integrations

Azure DevOps

- **What it does:** BlueLine receives a webhook when any PR is opened or updated. It reads the PR diff, posts inline review comments, creates fix branches, and opens draft PRs for security fixes.
- **What you need to configure:** One webhook in your ADO project settings pointing to the BlueLine API gateway. A service account with Code Read + Pull Request Threads Read/Write permissions.

Fortify SSC

- **What it does:** WATCHTOWER polls the Fortify SSC REST API for new findings. BULWARK reads finding details. STEWARD writes suppression decisions back to Fortify.
- **What you need to provide:** Fortify SSC API URL + API token with read access to your project. This is the **biggest blocker** for the Security Loop — without this, WATCHTOWER cannot connect.

Azure Key Vault

- **What it does:** TIMELINE reads certificate metadata (expiry dates). COURIER writes renewed certificate files. HARBOUR reads them for deployment.
- **What you need to provide:** Key Vault access policy granting BlueLine's Managed Identity `Certificate Get`, `Certificate List`, `Certificate Import` permissions.

IIS / Azure App Service

- **What it does:** HARBOUR connects to IIS servers via WinRM (remote PowerShell) to install certificates and update HTTPS bindings. For App Service, it uses the Azure SDK.
- **What you need to provide:** WinRM enabled on all IIS target servers. A service account with rights to import PFX certificates and modify IIS bindings remotely.

Microsoft Teams

- **What it does:** HARBOUR sends an approval card to a Teams channel before deploying any certificate to Production. Human clicks Approve or Reject directly in Teams.
- **What you need to provide:** An incoming webhook URL for the Teams channel you want to receive approval requests.

Certificate Authority (CA)

- **What it does:** COURIER calls the CA API to submit certificate renewal requests and download the issued certificate.
- **What you need to provide:** For internal certificates — C&M portal API access. For external certificates — DigiCert (or equivalent) API key. This is the **second biggest blocker** for the Certificate Loop.

---

8. Human Control Points

BlueLine is designed so that **no critical action ever happens without a human approving it first.**

| Action | Who approves | How | |---|---|---| | Merging a code review finding into the codebase | Any authorised PR reviewer | Standard ADO PR approval — no change to your existing process | | Merging a FORGE-generated security fix PR | Engineering lead or senior developer | FORGE creates a *draft* PR — it cannot be merged until a human approves it | | Deploying a certificate to Dev and QA | Automatic — no approval needed | Low risk environments, automated for speed | | Deploying a certificate to Production | Named approver | Teams card — human clicks Approve before HARBOUR proceeds | | Suppressing a Fortify finding as false positive | BULWARK logs reason; InfoSec reviews | STEWARD audit log reviewed periodically by InfoSec team |

If an agent is unsure about anything — confidence below threshold — it stops, does not act, and sends a notification to the team. It never guesses.

---

9. Getting Started — What We Need From You

Before BlueLine can be fully activated in your environment, the following access and configuration items are needed from your team:

Immediate (Quality Gate — ready in 1–2 weeks)

| Item | Who provides it | Purpose | |---|---|---| | Azure DevOps webhook configuration | Your DevOps team | Triggers BlueLine when a PR is opened | | ADO service account | Your DevOps team | BlueLine reads PRs and posts comments | | Azure Function App provisioned | Your platform team | Runtime for Quality Gate agents | | Azure OpenAI endpoint + key | Your Azure admin | Powers all AI reasoning |

Short Term (Security Loop — 3–4 weeks after access granted)

| Item | Who provides it | Purpose | |---|---|---| | **Fortify SSC API URL + Token** | InfoSec team | WATCHTOWER cannot run without this | | Azure Service Bus namespace | Your platform team | Agent-to-agent messaging | | ADO service account with branch/PR creation rights | Your DevOps team | FORGE creates fix branches |

Short-Medium Term (Certificate Loop — 4–6 weeks after access granted)

| Item | Who provides it | Purpose | |---|---|---| | **C&M portal API access** | InfoSec / Ops | COURIER cannot automate internal cert requests without this | | **Azure Key Vault access** | Your platform team | REGENT and TIMELINE read cert metadata | | **WinRM enabled on IIS servers** | Ops team | HARBOUR needs remote PowerShell to install certs | | Service account with cert permissions | Ops team | Import PFX and update IIS bindings | | Teams webhook URL | Any Teams admin | HARBOUR sends Prod approval notifications here |

---

10. Operating the System Day-to-Day

For Developers

Nothing changes. Open a PR as normal. BlueLine comments appear within 45 seconds. Review the comments — dismiss false positives, address real findings, and proceed with your normal review and merge process.

For Engineering Leads

- **ASCENT summary comments** on PRs show risk score and recommendation at a glance.
- **FORGE draft PRs** appear in your ADO backlog as security fix work. Review, test, and approve like any other PR.
- Periodically review the **ASCENT false positive report** — if a rule is flagging too often, it can be tuned.

For InfoSec

- **STEWARD audit logs** are in Azure Blob Storage — access via Azure Portal or Storage Explorer.
- **BULWARK classification decisions** are logged with reasoning — you can challenge or override any classification.
- **Fortify suppression decisions** made by BlueLine are recorded with a rationale — review periodically to validate.

For Operations

- **TIMELINE sends an alert** when any certificate enters the 30-day renewal window — you will receive this as a notification even if the automated process handles it.
- **HARBOUR sends a Teams card** before every Production certificate deployment — you or your approver clicks Approve.
- **Certificate inventory** is maintained in Azure Table Storage — accessible as a simple dashboard.

Shadow Mode (Recommended for First 2 Sprints)

For the first two sprints after deployment, we recommend running the Quality Gate agents in **shadow mode** — they review every PR and generate findings, but post **no comments** to the PR. All output goes to the logs. This lets your engineering lead validate accuracy before developers see any comments. Only switch to live mode after you are satisfied with the output quality.

---

11. Monitoring and Alerts

All agent activity is visible in **Azure Monitor + Log Analytics** under the `blueline-law` workspace.

What Is Monitored

| Event | Alert | Who Gets It | |---|---|---| | Any agent fails or throws an unhandled error | Immediate alert | BlueLine support channel | | Quality Gate agents not responding to PR webhooks | Alert after 5 minutes | DevOps team | | Certificate expiring within 7 days (emergency threshold) | Immediate alert | Ops team + InfoSec | | FORGE creates a CRITICAL fix PR | Notification | Engineering lead | | HARBOUR Prod deployment approved and completed | Confirmation | Ops team | | BULWARK false positive rate exceeds 20% for any rule | Weekly report | Engineering lead |

Key Dashboards (Azure Portal)

| Dashboard | Shows | |---|---| | Quality Gate dashboard | PRs reviewed per day, finding categories, false positive rate, ASCENT recommendations breakdown | | Security dashboard | Open findings by severity,

MTTR (Mean Time to Remediate), FORGE PR status | | Certificate dashboard | All certificates, expiry dates, renewal status per environment | | Audit log viewer | Full STEWARD log — every agent action with timestamp and reasoning |

---

12. Cost Estimate

BlueLine runs on your Azure infrastructure. Estimated ongoing monthly cost:

| Component | Estimated Cost / Month | |---|---| | Azure Functions (3 Function Apps) | ~$20–40 | | Azure Service Bus | ~$5–10 | | Azure Storage (state + audit logs) | ~$5–10 | | Azure Key Vault operations | ~$2–5 | | Azure Monitor + Log Analytics | ~$10–20 | | **Claude / Azure OpenAI token usage** | **~$30–80** (for 20–30 PRs/day + daily security + cert scans) | | Azure API Management | ~$10–15 | | **Total** | **~$80–180 / month** |

No per-seat licensing. No ongoing subscription to LTM. You own the code.

---

13. Handover Checklist

Use this list to confirm everything is in place before full go-live:

Environment

- [ ] Azure resource groups provisioned (`blueline-rg-shared`, `blueline-rg-quality`, `blueline-rg-security`, `blueline-rg-certloop`)
- [ ] Azure Function Apps deployed with latest agent code
- [ ] Azure Key Vault provisioned and access policies set
- [ ] Azure Service Bus namespace created with correct topics
- [ ] Azure Storage Account created with audit log container
- [ ] Azure Monitor alerts configured

Quality Gate

- [ ] ADO webhook configured and tested (sends on PR create + update)
- [ ] CLARION and LUMEN loaded with your DAS/CDAS coding standards document
- [ ] Shadow mode enabled for first 2 sprints
- [ ] Engineering lead has access to shadow mode output logs

Security Loop

- [ ] Fortify SSC API URL and token stored in Key Vault
- [ ] WATCHTOWER scan schedule confirmed
- [ ] FORGE ADO service account has branch creation + PR creation rights
- [ ] STEWARD audit log container confirmed immutable (WORM policy enabled)
- [ ] InfoSec team briefed on STEWARD log access and review process

Certificate Loop

- [ ] All existing certificates imported into Azure Key Vault
- [ ] C&M portal API credentials stored in Key Vault
- [ ] WinRM enabled on all IIS target servers
- [ ] Cert deployment service account provisioned
- [ ] Teams webhook URL configured for HARBOUR approval notifications
- [ ] TIMELINE tested in read-only mode — expiry detection validated

Handover

- [ ] Code repository access transferred to your team (`https://github.com/sagu25/blueline.git`)
- [ ] All documents shared and reviewed
- [ ] Runbook walkthrough completed with your DevOps team
- [ ] On-call contact established for first 4 weeks post go-live

---

14. Support and Ownership After Handover

You Own the Code

Everything is in your repository. There is no vendor lock-in, no licence dependency, and no mandatory ongoing relationship with LTM. Your team can read, modify, and extend every agent.

What Your Team Needs to Maintain It

- A Python developer to maintain agent logic (any .NET/Python developer can do this)
- A tech lead or architect to update the coding standards document in CLARION/LUMEN when your standards evolve
- A DevOps engineer to manage the Azure Function Apps and deployment pipeline

Extending BlueLine

Each agent is an independent Python class. To add a new capability — for example, reviewing MAUI mobile code, or integrating a new CA — you update or add one agent file. The infrastructure and communication layer does not need to change.

Optional LTM Support

LTM can provide a support arrangement for the first 3–6 months post-delivery for:

- Bug fixes and rule tuning
- Extension to new repositories or environments
- Adding new agent capabilities
- Training your team on agent development

This is optional — the system is fully documented and self-contained.

---

15. Frequently Asked Questions

Q: What happens if Azure OpenAI goes down? Agents fail gracefully. PRs remain open and reviewable by humans normally. STEWARD logs the failure. An alert fires to your Teams channel. No PR is blocked — the pipeline degrades to manual review, it does not break.

Q: Can we change the coding rules CLARION uses? Yes. Rules live in `docs/das_review_standards.md`. Edit the file, redeploy the Function App — CLARION picks up the new rules immediately. No code change required.

Q: What if the false positive rate is too high? Use shadow mode for the first 2 sprints to tune before going live. After go-live, ASCENT tracks every comment a developer marks as false positive. When any rule crosses 20% false positive rate, it flags it for your engineering lead to review.

Q: Can agents review code in other languages — Ember, MAUI, Python? Current scope is C# (.NET) and TypeScript (Angular). MAUI is C# so .NET rules apply automatically. Ember and Python can be added by updating the system prompt — no infrastructure change. This is a Phase 2 addition.

Q: Can different repos have different rules? Yes. Each Function App deployment can load a different standards document per repository or team. This is configured as an environment variable.

Q: Who maintains BlueLine after handover? Your team, using the documentation and runbook provided. LTM optional support is available but not required.

Q: What is the difference between BlueLine and GitHub Copilot / Spec Kit? GitHub Copilot and Spec Kit help individual developers write new code faster — they are triggered manually by a developer.

BlueLine runs automatically on everything your team builds, without anyone needing to remember to use it. They solve different problems and can coexist.

Q: Can HARBOUR integrate with our ServiceNow change management process? Yes — HARBOUR can be extended to create a Change Request in ServiceNow before a Production deployment and wait for approval there instead of (or in addition to) the Teams card. This is a scoped extension we can deliver as part of an optional support engagement.

Q: How do we measure whether BlueLine is working? Three metrics to track from day one:

1. **PR review cycle time** — time from PR opened to first human reviewer comment (should drop significantly)
2. **Security finding resolution time** — time from Fortify finding created to fix merged
3. **Certificate expiry incidents** — target is zero

Project BlueLine — Handover Document v1.0 LTM AI-Led Engineering Team | April 2026 Repository: https://github.com/sagu25/blueline.git